In this chapter, you'll see:

- How to return, raise, or yield a value from your double
- How to supply custom behavior for your double
- How to ensure your double is called with the right arguments
- How to make sure your double is called the right number of times and in the right order

CHAPTER 14

Customizing Test Doubles

Now that you've got a handle on the basic types of test doubles and when to use them, we're going to dive into the rspec-mocks API for a bit. Our goal isn't to give you an exhaustive API reference here; that's what the docs are for.[1]

Instead, we're going to show you the basics, and then give a few recipes for specific situations.

Configuring Responses

Since a test double is meant to stand in for a real object, it needs to act like one. You need to be able to configure how it *responds* to the code calling it.

When you `allow` or `expect` a message on a test double without specifying how it responds, RSpec provides a simple implementation that just returns `nil`. Your test doubles will often need to do something more interesting: return a given value, raise an error, yield to a block, or throw a symbol. RSpec provides ways for your doubles to do each of these:

14-customizing-test-doubles/01/configuring_responses.rb

```
allow(double).to receive(:a_message).and_return(a_return_value)
allow(double).to receive(:a_message).and_raise(AnException)
allow(double).to receive(:a_message).and_yield(a_value_to_a_block)
allow(double).to receive(:a_message).and_throw(:a_symbol, optional_value)
allow(double).to receive(:a_message) { |arg| do_something_with(arg) }

# These last two are just for partial doubles:
allow(object).to receive(:a_message).and_call_original
allow(object).to receive(:a_message).and_wrap_original { |original| }
```

Now, let's look at a couple of specific situations you may run into when you're specifying how your test doubles will behave.

1. https://relishapp.com/rspec/rspec-mocks/v/3-6/docs/configuring-responses